ECE 526L Spring 2025 Lab 9

Parameterized Synchronous FIFO



Muhammad Umar
CSUN ID: 204533244

Instructor: Prof. Orod Haghighiara

I hereby attest that this lab report is entirely my own work. I have not copied either code or text from anyone, nor have I allowed or will I allow anyone to copy my work.

Name (printed): **Muhammad Umar**

Name (signed):  Date: 05/02/2025

INTRODUCTION

This lab focused on the design, implementation, and thorough verification of a parameterized synchronous **FIFO** (First-In, First-Out) memory module using SystemVerilog. FIFO buffers are essential in digital systems where data needs to be temporarily stored and passed between modules operating at different speeds or on different or same clock domains. The FIFO structure guarantees data integrity by maintaining the order in which data was written, allowing it to be read back in the exact same sequence.

The primary objective of this experiment is to build a flexible FIFO that could be adapted for various sizes and configurations. We utilized **parameterization** to define the data **width** and **depth** of the FIFO, enabling scalability and reuse across projects. The FIFO supports standard operational flags such as **empty**, **full**, **almost_empty**, and **almost_full** to indicate the buffer's state, and also includes error handling through **overflow** and **underflow** flags. The **count** signal tracks the number of items in the buffer, while the **valid** signal confirms the correctness of read operations.

Beyond standard FIFO behavior, the design has extended to support two advanced features through conditional compilation using Verilog preprocessor directives:

First Word Fall Through (FWFT): Enabled via ifdef FWFT, this feature allows the first written word to appear at the output without requiring a **read enable** signal, reducing access latency.

The accompanying testbench has developed to rigorously evaluate the FIFO's functionality, covering normal operation, boundary cases, simultaneous read/write scenarios, and the specific behaviors introduced by FWFT mode. Simulation logs and waveforms were captured to verify correct operation under all tested conditions, ensuring the design meets functional requirements and is robust against edge cases.

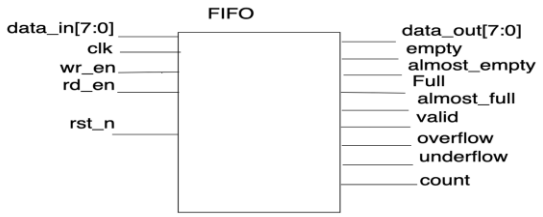


Figure 9.1 Block Diagram of FIFO

METHODOLOGY AND TEST PLAN

FIFO DESIGN OVERVIEW

The FIFO module is designed to be flexible and scalable through the use of parameters, namely **DATA_WIDTH** and **DEPTH**, which are set to 8 and 32, respectively. This allows the design to accommodate varying sizes of data and storage requirements. Internally, it uses a memory array to store incoming data along with two pointers (**wr_ptr** and **rd_ptr**) to manage write and read operations circularly. A **count** signal keeps track of how many data items are currently stored.

The module provides a complete set of status signals. These include **full** and **almost_full** to indicate nearing or complete storage capacity, **empty** and **almost_empty** to signal data depletion, and **overflow** and **underflow** flags to detect invalid write or read attempts,

respectively. The **valid** output denotes a successful read, and data_out provides the data read from the FIFO.

To support advanced functionality, two conditional modes have been included using Verilog **ifdef** macros. In **Normal Mode**, the data_out is updated only when a read enable (**rd_en**) signal is active and the FIFO is not empty. In contrast, **First Word Fall Through (FWFT) Mode**, enabled using **ifdef FWFT**, allows the first available data to preload to **data_out** as soon as the FIFO is non-empty, even before rd_en is asserted, thereby reducing read latency.

TESTBENCH STRATEGY

A thorough Verilog testbench (FIFO_tb.sv) was developed to verify the FIFO under the following scenarios:

1.Write until Full + Overflow Test

The FIFO was checked by writing data to all available memory locations sequentially. As the internal counter approached its maximum capacity, the **almost_full** flag was asserted to indicate the buffer was nearing its limit. Upon reaching the maximum number of entries, the **full** flag was activated. Any additional write attempts beyond this point resulted in the **overflow** flag being asserted, correctly indicating that the FIFO could no longer accept new data. These behaviors, including all flag transitions and overflow conditions, were captured and confirmed through the simulation log and waveform, which are both included in the appendices below.

```
================================================================
Write to FIFO more than its capacity to test full and overflow
================================================================

Write: 0, Count: 1, Full: 0, Almost_Full: 0, Overflow: 0
Write: 1, Count: 2, Full: 0, Almost_Full: 0, Overflow: 0
Write: 2, Count: 3, Full: 0, Almost_Full: 0, Overflow: 0
Write: 3, Count: 4, Full: 0, Almost_Full: 0, Overflow: 0
Write: 4, Count: 5, Full: 0, Almost_Full: 0, Overflow: 0
Write: 5, Count: 6, Full: 0, Almost_Full: 0, Overflow: 0
Write: 6, Count: 7, Full: 0, Almost_Full: 0, Overflow: 0
Write: 7, Count: 8, Full: 0, Almost_Full: 0, Overflow: 0
Write: 8, Count: 9, Full: 0, Almost_Full: 0, Overflow: 0
Write: 9, Count: 10, Full: 0, Almost_Full: 0, Overflow: 0
Write: 10, Count: 11, Full: 0, Almost_Full: 0, Overflow: 0
Write: 11, Count: 12, Full: 0, Almost_Full: 0, Overflow: 0
Write: 12, Count: 13, Full: 0, Almost_Full: 0, Overflow: 0
Write: 13, Count: 14, Full: 0, Almost_Full: 0, Overflow: 0
Write: 14, Count: 15, Full: 0, Almost_Full: 0, Overflow: 0
Write: 15, Count: 16, Full: 0, Almost_Full: 0, Overflow: 0
Write: 16, Count: 17, Full: 0, Almost_Full: 0, Overflow: 0
Write: 17, Count: 18, Full: 0, Almost_Full: 0, Overflow: 0
Write: 18, Count: 19, Full: 0, Almost_Full: 0, Overflow: 0
Write: 19, Count: 20, Full: 0, Almost_Full: 0, Overflow: 0
Write: 20, Count: 21, Full: 0, Almost_Full: 0, Overflow: 0
Write: 21, Count: 22, Full: 0, Almost_Full: 0, Overflow: 0
Write: 22, Count: 23, Full: 0, Almost_Full: 0, Overflow: 0
Write: 23, Count: 24, Full: 0, Almost_Full: 0, Overflow: 0
Write: 24, Count: 25, Full: 0, Almost_Full: 0, Overflow: 0
Write: 25, Count: 26, Full: 0, Almost_Full: 0, Overflow: 0
Write: 26, Count: 27, Full: 0, Almost_Full: 0, Overflow: 0
Write: 27, Count: 28, Full: 0, Almost_Full: 0, Overflow: 0
Write: 28, Count: 29, Full: 0, Almost_Full: 0, Overflow: 0
Write: 29, Count: 30, Full: 0, Almost_Full: 0, Overflow: 0
Write: 30, Count: 31, Full: 0, Almost_Full: 1, Overflow: 0

*** Warning: FIFO is ALMOST FULL! ***

Write: 31, Count: 32, Full: 0, Almost_Full: 1, Overflow: 0
```

Figure 9.2 Log of Writing to FIFO and Almost Full Flag Asserted

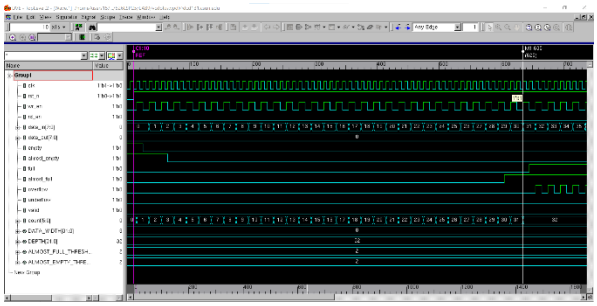


Figure 9.3 Waveform of Writing to FIFO and Almost Full Flag Asserted

As the FIFO fills, the **count** increases with each write until it reaches 32, at which point the full flag is asserted and further writes are blocked. The count remains stable, and the **overflow** flag is asserted on each invalid write attempt, indicating that no more data can be accepted. This behavior is clearly shown in the log snapshot below, where repeated writes at count = 32 consistently trigger the overflow signal without increasing the count.

```
Write: 29, Count: 30, Full: 0, Almost_Full: 0, Overflow: 0
Write: 30, Count: 31, Full: 0, Almost_Full: 1, Overflow: 0

*** Warning: FIFO is ALMOST FULL! ***

Write: 31, Count: 32, Full: 0, Almost_Full: 1, Overflow: 0

*** Warning: FIFO is ALMOST FULL! ***

Write: 32, Count: 32, Full: 1, Almost_Full: 1, Overflow: 1
*** FIFO is FULL! No more data can be written! ***
Write: 33, Count: 32, Full: 1, Almost_Full: 1, Overflow: 1
*** FIFO is FULL! No more data can be written! ***
Write: 34, Count: 32, Full: 1, Almost_Full: 1, Overflow: 1
*** FIFO is FULL! No more data can be written! ***
Write: 35, Count: 32, Full: 1, Almost_Full: 1, Overflow: 1
*** FIFO is FULL! No more data can be written! ***
Write: 36, Count: 32, Full: 1, Almost_Full: 1, Overflow: 1
*** FIFO is FULL! No more data can be written! ***
```

Figure 9.4 Log of Writing to FIFO, Full, Overflow, Almost Full Flag Asserted

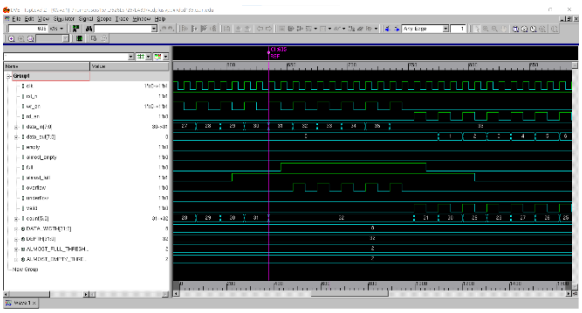


Figure 9.5 Waveform of Writing to FIFO, Full, Almost Full, Overflow Flag Asserted

2. Read until Empty+Underflow Test

The FIFO was sequentially read from a full state until it became completely empty. As the internal **count** decreased, the **almost_empty** flag activated when the remaining data fell below the configured threshold. Upon reaching zero entries, the **empty** flag was asserted. Any further read attempts beyond this point triggered the **underflow** flag, indicating invalid access to an empty buffer. The **valid** signal correctly deasserted during these illegal reads. These transitions including **almost_empty**, **empty**, and **underflow** were clearly observed in both the simulation logs and the waveform captures provided below, verifying proper status signaling during buffer depletion.

```
================================================================
Read from FIFO until empty to test empty and underflow
================================================================

Read: 0, Data_out: 0, Valid: 1, Count: 31, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 1, Data_out: 1, Valid: 1, Count: 30, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 2, Data_out: 2, Valid: 1, Count: 29, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 3, Data_out: 3, Valid: 1, Count: 28, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 4, Data_out: 4, Valid: 1, Count: 27, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 5, Data_out: 5, Valid: 1, Count: 26, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 6, Data_out: 6, Valid: 1, Count: 25, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 7, Data_out: 7, Valid: 1, Count: 24, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 8, Data_out: 8, Valid: 1, Count: 23, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 9, Data_out: 9, Valid: 1, Count: 22, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 10, Data_out: 10, Valid: 1, Count: 21, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 11, Data_out: 11, Valid: 1, Count: 20, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 12, Data_out: 12, Valid: 1, Count: 19, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 13, Data_out: 13, Valid: 1, Count: 18, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 14, Data_out: 14, Valid: 1, Count: 17, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 15, Data_out: 15, Valid: 1, Count: 16, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 16, Data_out: 16, Valid: 1, Count: 15, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 17, Data_out: 17, Valid: 1, Count: 14, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 18, Data_out: 18, Valid: 1, Count: 13, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 19, Data_out: 19, Valid: 1, Count: 12, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 20, Data_out: 20, Valid: 1, Count: 11, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 21, Data_out: 21, Valid: 1, Count: 10, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 22, Data_out: 22, Valid: 1, Count: 9, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 23, Data_out: 23, Valid: 1, Count: 8, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 24, Data_out: 24, Valid: 1, Count: 7, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 25, Data_out: 25, Valid: 1, Count: 6, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 26, Data_out: 26, Valid: 1, Count: 5, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 27, Data_out: 27, Valid: 1, Count: 4, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 28, Data_out: 28, Valid: 1, Count: 3, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 29, Data_out: 29, Valid: 1, Count: 2, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 30, Data_out: 30, Valid: 1, Count: 1, Empty: 0, Almost_Empty: 1, Underflow: 0

*** Warning: FIFO is ALMOST EMPTY! ***

Read: 31, Data_out: 31, Valid: 1, Count: 0, Empty: 0, Almost_Empty: 1, Underflow: 0
```

```
Read: 26, Data_out: 26, Valid: 1, Count: 5, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 27, Data_out: 27, Valid: 1, Count: 4, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 28, Data_out: 28, Valid: 1, Count: 3, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 29, Data_out: 29, Valid: 1, Count: 2, Empty: 0, Almost_Empty: 0, Underflow: 0
Read: 30, Data_out: 30, Valid: 1, Count: 1, Empty: 0, Almost_Empty: 1, Underflow: 0

*** Warning: FIFO is ALMOST EMPTY! ***

Read: 31, Data_out: 31, Valid: 1, Count: 0, Empty: 0, Almost_Empty: 1, Underflow: 0

*** Warning: FIFO is ALMOST EMPTY! ***

Read: 32, Data_out: 31, Valid: 0, Count: 0, Empty: 1, Almost_Empty: 1, Underflow: 1
*** ERROR: FIFO UNDERFLOW occurred! Tried to read when empty! ***
Read: 33, Data_out: 31, Valid: 0, Count: 0, Empty: 1, Almost_Empty: 1, Underflow: 1
*** ERROR: FIFO UNDERFLOW occurred! Tried to read when empty! ***
Read: 34, Data_out: 31, Valid: 0, Count: 0, Empty: 1, Almost_Empty: 1, Underflow: 1
*** ERROR: FIFO UNDERFLOW occurred! Tried to read when empty! ***
Read: 35, Data_out: 31, Valid: 0, Count: 0, Empty: 1, Almost_Empty: 1, Underflow: 1
*** ERROR: FIFO UNDERFLOW occurred! Tried to read when empty! ***
Read: 36, Data_out: 31, Valid: 0, Count: 0, Empty: 1, Almost_Empty: 1, Underflow: 1
*** ERROR: FIFO UNDERFLOW occurred! Tried to read when empty! ***
```

Figure 9.6 Log of Reading from FIFO, Empty, Almost Empty Flag Asserted

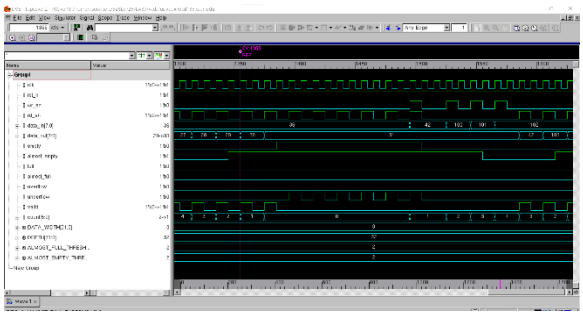


Figure 9.7 Waveform of Reading from FIFO, Empty, Almost Empty, Underflow Flag Asserted

3. Simultaneous Write and Read

During the simultaneous write and read test, both **wr_en** and **rd_en** were asserted in the same clock cycle to evaluate whether the

FIFO could correctly manage concurrent access. The results confirmed that the internal write and read pointers operated independently and accurately, allowing a new value to be written while a different, previously stored value was read out. Specifically, while data_in = 42 was written, data_out returned the previously written value 31, validating that the FIFO maintained correct sequential behavior without overwriting or misaligning data. This behavior was clearly captured in the console output and waveform snapshot shown below.

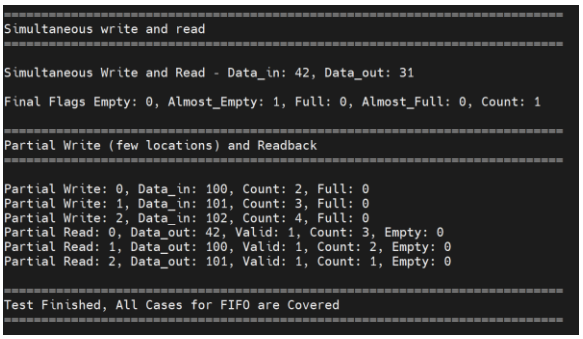


Figure 9.8 Log of Simultaneous Read and Write from FIFO

4. Partial Write and Readback

In the partial write and readback scenario, three values (**100**, **101**, and **102**) were written to the FIFO and subsequently read back. This test verified that the FIFO correctly stored and retrieved a limited number of entries without reaching full capacity. Each read operation returned the expected value in sequence, confirming data integrity and proper pointer handling. The count value accurately reflected the number of stored entries throughout the operations, and the valid and empty flags responded correctly to each read. These behaviors are clearly illustrated in the log snapshot provided below.

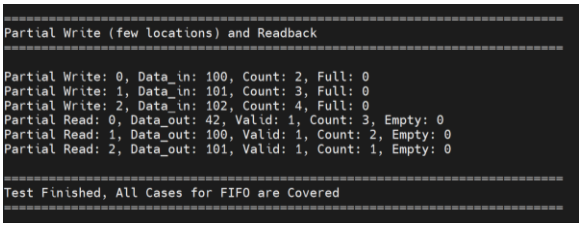


Figure 9.9 Log of Partial Read and Write from FIFO

5. FWFT-Specific Preload Test

To verify the First Word Fall Through (**FWFT**) behavior, the FIFO was tested by writing a single value while keeping the read enable signal (**rd_en**) deasserted. As soon as the FIFO became non-empty, the written data (**55**) appeared on **data_out** immediately, even before a read request was issued. This confirmed that the FIFO correctly preloaded the first word onto the output without waiting for **rd_en**, enabling zero-latency access to the data. When rd_en was later asserted, the same value was still available and the valid signal confirmed the successful read. This behavior is clearly demonstrated in the simulation log snapshot shown below, illustrating that FWFT mode operated exactly as intended.

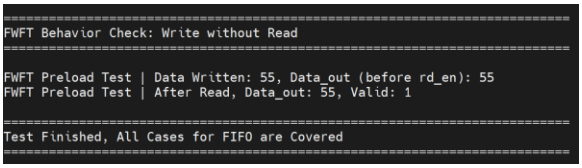


Figure 9.10 Log of FWFT FIFO

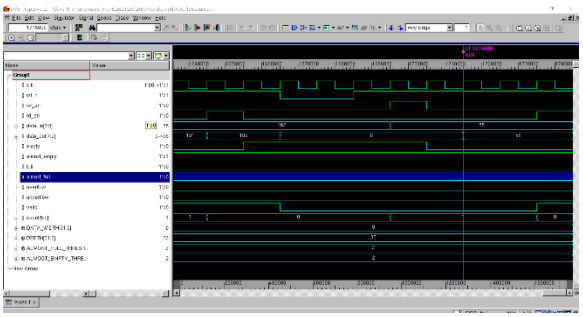


Figure 9.11 Waveform of FWFT FIFO

5. FWFT Mode (Enabled):

- Verified preload behavior by writing a value and checking data_out before rd_en.
- Observed data_out = 55 immediately after write, even without read enable
- Confirmed proper behavior in waveform and console output.

RESULTS

1. Write to Full / Overflow:

- FIFO correctly asserted almost_full at count 30 and full at count 32.
- Additional writes set overflow = 1 and blocked data input.
- Console logs confirmed each status with appropriate warnings

2. Read to Empty / Underflow:

- Read operations reduced count from 32 to 0.
- almost_empty activated at count 2; empty and underflow behaved as expected.

3. Simultaneous Write/Read:

- Demonstrated that read and write pointers updated concurrently.
- data_out was not overwritten by data_in on the same cycle.

4. Partial Write/Read:

- Successfully stored and retrieved three entries (100, 101, 102).
- Validated intermediate FIFO states and valid signal.

EXTRA CREDIT:

First Word Fall Through (FWFT)

Using **ifdef FWFT**, the FIFO supports zero-latency access to the first entry when non-empty. A preload logic block places the first word on data_out automatically. The testbench conditionally enables FWFT verification and compares it to normal behavior by checking **data_out** before and after asserting **rd_en**.

- **Normal Mode:** data_out updates only after rd_en.
- **FWFT Mode:** data_out reflects oldest value once FIFO is non-empty, before rd_en.

Asynchronous FIFO Proposal

Asynchronous functionality can be achieved by separating the write and read clocks (wr_clk and rd_clk) and synchronizing the read and write pointers across clock domains. This requires using techniques like Gray code pointer conversion and multi-stage flip-flop synchronization to prevent metastability. While not implemented in this lab, this

approach forms the basis for creating a robust asynchronous FIFO in future work.

CONCLUSION

This lab successfully demonstrated the design and validation of a flexible, parameterized FIFO memory with enhanced features like **FWFT**. The testbench verified all flag behaviors, read/write operations, and simultaneous transactions. Conditional compilation was used to toggle between normal and FWFT modes, and a baseline asynchronous FIFO architecture was introduced.

Waveform and log file analysis confirmed correctness across all tests. The design aligns with industry standards and digital design best practices, ensuring robustness and extensibility.

APPENDIX

Appendix A: FIFO.sv

```
/*****************************************************************************
*** EE 526 L Experiment #9            Muhammad Umar, Spring, 2025     ***
***                                                                   ***
*** Experiment #9 Parameterized Synchronous FIFO                      ***
***                                                                   ***
*** Filename: FIFO.sv                                                 ***
***                          Created by Muhammad Umar, 05/01/2025     ***
***                                                                   ***
*** This FIFO module implements a parameterized synchronous FIFO      ***
*** with support for First Word Fall Through (FWFT) using ifdef,      ***
*** along with standard status flags and overflow/underflow detection.***
*****************************************************************************/
`timescale 1ns / 1ps


module FIFO #(
    parameter DATA_WIDTH          = 8 ,
    parameter DEPTH               = 32,
    parameter ALMOST_FULL_THRESH  = 2 ,
    parameter ALMOST_EMPTY_THRESH = 2
) (
    input  logic                    clk        ,
    input  logic                    rst_n      ,
    input  logic                    wr_en      ,
    input  logic                    rd_en      ,
    input  logic [ DATA_WIDTH-1:0] data_in     ,
    output logic [ DATA_WIDTH-1:0] data_out    ,
    output logic                    empty      ,
    output logic                    almost_empty,
    output logic                    full       ,
    output logic                    almost_full ,
    output logic                    overflow   ,
    output logic                    underflow  ,
    output logic                    valid      ,
    output logic [$clog2(DEPTH):0] count
);

    logic [   DATA_WIDTH-1:0] mem   [0:DEPTH-1];
    logic [$clog2(DEPTH)-1:0] rd_ptr, wr_ptr;

    `ifdef FWFT
        logic preload;
    `endif
```

```
always_ff @(posedge clk) begin
    if (!rst_n) begin
        rd_ptr       <= 0;
        wr_ptr       <= 0;
        count        <= 0;
        empty        <= 1;
        full         <= 0;
        almost_empty <= 1;
        almost_full  <= 0;
        overflow     <= 0;
        underflow    <= 0;
        valid        <= 0;
        data_out     <= 0;
        `ifdef FWFT
            preload <= 0;
        `endif
    end else begin
        // Write operation
        if (wr_en && !full) begin
            mem[wr_ptr] <= data_in;
            wr_ptr      <= (wr_ptr + 1) % DEPTH;
            count       <= count + 1;
            overflow    <= 0;
        end else if (wr_en && full) begin
            overflow <= 1;
        end else begin
            overflow <= 0;
        end


        `ifdef FWFT
            if (!rd_en && !empty && !valid) begin
                data_out <= mem[rd_ptr];
                preload  <= 1;
            end
        `endif

        // Read operation
        if (rd_en && !empty) begin
            data_out <= mem[rd_ptr];
            rd_ptr   <= (rd_ptr + 1) % DEPTH;
            count    <= count - 1;
            valid    <= 1;
```

```
                `ifdef FWFT
                    preload <= 0;
                `endif
                underflow <= 0;
            end else if (rd_en && empty) begin
                underflow <= 1;
        valid    <= 0;
            end else begin
                underflow <= 0;
            end
            // Flags
            empty        <= (count == 0);
            almost_empty <= (count <= ALMOST_EMPTY_THRESH);
            full         <= (count == DEPTH);
            almost_full  <= (count >= (DEPTH - ALMOST_FULL_THRESH));
        end
    end
endmodule
```

Appendix B: multiplier.v

```
/*************************************************************************
*** EE 526 L Experiment #9            Muhammad Umar, Spring, 2025     ***
***                                                                   ***
*** Experiment #9 Parameterized Synchronous FIFO                      ***
***                                                                   ***
*** Filename: FIFO_tb.sv                                              ***
***                        Created by Muhammad Umar, 05/01/2025       ***
***                                                                   ***
*** This SystemVerilog testbench verifies the functionality of the    ***
*** parameterized FIFO module by testing overflow, underflow, flag    ***
*** transitions, simultaneous operations, and First Word Fall Through ***
*** (FWFT) behavior conditionally using ifdef FWFT.
*************************************************************************/

module FIFO_tb ();

    parameter DATA_WIDTH          = 8 ;
    parameter DEPTH               = 32;
    parameter ALMOST_FULL_THRESH  = 2 ;
    parameter ALMOST_EMPTY_THRESH = 2 ;

    logic                   clk         ;
    logic                   rst_n       ;
    logic                   wr_en       ;
    logic                   rd_en       ;
    logic [ DATA_WIDTH-1:0] data_in     ;
    logic [ DATA_WIDTH-1:0] data_out    ;
    logic                   empty       ;
    logic                   almost_empty;
    logic                   full        ;
    logic                   almost_full ;
    logic                   overflow    ;
    logic                   underflow   ;
    logic                   valid       ;
    logic [$clog2(DEPTH):0] count       ;

    FIFO #(
        .DATA_WIDTH         (DATA_WIDTH         ),
        .DEPTH              (DEPTH              ),
        .ALMOST_FULL_THRESH (ALMOST_FULL_THRESH ),
        .ALMOST_EMPTY_THRESH(ALMOST_EMPTY_THRESH)
    ) dut (
        .clk         (clk         ),
        .rst_n       (rst_n       ),
```

```
        .wr_en       (wr_en       ),
        .rd_en       (rd_en       ),
        .data_in     (data_in     ),
        .data_out    (data_out    ),
        .empty       (empty       ),
        .almost_empty(almost_empty),
        .full        (full        ),
        .almost_full (almost_full ),
        .overflow    (overflow    ),
        .underflow   (underflow   ),
        .valid       (valid       ),
        .count       (count       )
    );

    // Clock generation
    always #5 clk = ~clk;

    initial begin
        $vcdpluson;

        clk = 0;
        rst_n = 0;
        wr_en = 0;
        rd_en = 0;
        data_in = 0;

        #10;
        rst_n = 1;



$display("\n============================================================
==================");
        $display("Write to FIFO more than its capacity to test full
and overflow");

$display("============================================================
===============\n");

        for (int i = 0; i < (DEPTH + 5); i++) begin
            @(posedge clk);
            wr_en = 1;
            rd_en = 0;
            data_in = i;
```

```
            @(posedge clk);
            wr_en = 0;

            $display("Write: %0d, Count: %0d, Full: %b, Almost_Full:
%b, Overflow: %b", i, count, full, almost_full, overflow);

            if (full) begin
                $display("*** FIFO is FULL! No more data can be
written! ***");
            end else if (almost_full) begin
                $display("\n*** Warning: FIFO is ALMOST FULL!
***\n");
            end
        end

        wr_en = 0;


$display("\n=============================================================
==================");
        $display("Read from FIFO until empty to test empty and
underflow");

$display("=============================================================
================\n");

        for (int i = 0; i < (DEPTH + 5); i++) begin
            @(posedge clk);
            wr_en = 0;
            rd_en = 1;
            @(posedge clk);
            rd_en = 0;

            $display("Read: %0d, Data_out: %0d, Valid: %b, Count:
%0d, Empty: %b, Almost_Empty: %b, Underflow: %b", i, data_out, valid,
count, empty, almost_empty, underflow);

            if (underflow) begin
                $display("*** ERROR: FIFO UNDERFLOW occurred! Tried
to read when empty! ***");
            end else if (empty) begin
                $display("\n*** FIFO is EMPTY! No more data to read.
***\n");
            end else if (almost_empty) begin
```

```
                $display("\n*** Warning: FIFO is ALMOST EMPTY!
***\n");
            end
        end
        rd_en = 0;


$display("\n============================================================
==================");
        $display("Simultaneous write and read");

$display("============================================================
================\n");
        @(posedge clk);
        wr_en = 1;
        rd_en = 1;
        data_in = 42;
        @(posedge clk);
        wr_en = 0;
        rd_en = 0;
        $display("Simultaneous Write and Read - Data_in: %0d,
Data_out: %0d", data_in, data_out);

        @(posedge clk);
        $display("\nFinal Flags Empty: %b, Almost_Empty: %b, Full:
%b, Almost_Full: %b, Count: %0d", empty, almost_empty, full,
almost_full, count);


$display("\n============================================================
==================");
        $display("Partial Write (few locations) and Readback");

$display("============================================================
================\n");

        for (int j = 0; j < 3; j++) begin
            @(posedge clk);
            wr_en = 1;
            rd_en = 0;
            data_in = 100 + j;
            @(posedge clk);
            wr_en = 0;
```

```
            $display("Partial Write: %0d, Data_in: %0d, Count: %0d,
Full: %b", j, data_in, count, full);
        end

        for (int j = 0; j < 4; j++) begin
            @(posedge clk);
            wr_en = 0;
            rd_en = 1;
            @(posedge clk);
            rd_en = 0;
            $display("Partial Read: %0d, Data_out: %0d, Valid: %b,
Count: %0d, Empty: %b", j, data_out, valid, count, empty);
        end



        `ifdef FWFT
            @(posedge clk);
            rst_n = 0;
            repeat(2) begin
                @(posedge clk);
            end

            rst_n = 1;


$display("\n============================================================
==================");
            $display("FWFT Behavior Check: Write without Read");

$display("============================================================
================\n");

            @(posedge clk);
            data_in = 55;
            wr_en = 1;
            rd_en = 0;
            @(posedge clk);
            wr_en = 0;
            repeat(2) begin
                @(posedge clk);
            end
```

```
            $display("FWFT Preload Test | Data Written: %0d, Data_out
(before rd_en): %0d", data_in, data_out);

            // Now asserting rd_en to consume
            @(posedge clk);
            rd_en = 1;
            @(posedge clk);
            rd_en = 0;

            $display("FWFT Preload Test | After Read, Data_out: %0d,
Valid: %b", data_out, valid);
        `endif




$display("\n=============================================================
==================");
        $display("Test Finished, All Cases for FIFO are Covered");

$display("=============================================================
================\n");


        $finish;
    end

endmodule
```

Appendix C: FIFO.log

```
Command: /home/users15/mu832615/526LSP25/LAB9/./simv -l FIFO.log
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version U-2023.03-SP1-1_Full64; Runtime version U-2023.03-
SP1-1_Full64;  May  2 22:42 2025
VCD+ Writer U-2023.03-SP1-1_Full64 Copyright (c) 1991-2023 by
Synopsys Inc.

================================================================
======
Write to FIFO more than its capacity to test full and overflow
================================================================
======

Write: 0, Count: 1, Full: 0, Almost_Full: 0, Overflow: 0
Write: 1, Count: 2, Full: 0, Almost_Full: 0, Overflow: 0
Write: 2, Count: 3, Full: 0, Almost_Full: 0, Overflow: 0
Write: 3, Count: 4, Full: 0, Almost_Full: 0, Overflow: 0
Write: 4, Count: 5, Full: 0, Almost_Full: 0, Overflow: 0
Write: 5, Count: 6, Full: 0, Almost_Full: 0, Overflow: 0
Write: 6, Count: 7, Full: 0, Almost_Full: 0, Overflow: 0
Write: 7, Count: 8, Full: 0, Almost_Full: 0, Overflow: 0
Write: 8, Count: 9, Full: 0, Almost_Full: 0, Overflow: 0
Write: 9, Count: 10, Full: 0, Almost_Full: 0, Overflow: 0
Write: 10, Count: 11, Full: 0, Almost_Full: 0, Overflow: 0
Write: 11, Count: 12, Full: 0, Almost_Full: 0, Overflow: 0
Write: 12, Count: 13, Full: 0, Almost_Full: 0, Overflow: 0
Write: 13, Count: 14, Full: 0, Almost_Full: 0, Overflow: 0
Write: 14, Count: 15, Full: 0, Almost_Full: 0, Overflow: 0
Write: 15, Count: 16, Full: 0, Almost_Full: 0, Overflow: 0
Write: 16, Count: 17, Full: 0, Almost_Full: 0, Overflow: 0
Write: 17, Count: 18, Full: 0, Almost_Full: 0, Overflow: 0
Write: 18, Count: 19, Full: 0, Almost_Full: 0, Overflow: 0
Write: 19, Count: 20, Full: 0, Almost_Full: 0, Overflow: 0
Write: 20, Count: 21, Full: 0, Almost_Full: 0, Overflow: 0
Write: 21, Count: 22, Full: 0, Almost_Full: 0, Overflow: 0
Write: 22, Count: 23, Full: 0, Almost_Full: 0, Overflow: 0
Write: 23, Count: 24, Full: 0, Almost_Full: 0, Overflow: 0
Write: 24, Count: 25, Full: 0, Almost_Full: 0, Overflow: 0
Write: 25, Count: 26, Full: 0, Almost_Full: 0, Overflow: 0
Write: 26, Count: 27, Full: 0, Almost_Full: 0, Overflow: 0
Write: 27, Count: 28, Full: 0, Almost_Full: 0, Overflow: 0
Write: 28, Count: 29, Full: 0, Almost_Full: 0, Overflow: 0
```

```
Write: 29, Count: 30, Full: 0, Almost_Full: 0, Overflow: 0
Write: 30, Count: 31, Full: 0, Almost_Full: 1, Overflow: 0

*** Warning: FIFO is ALMOST FULL! ***

Write: 31, Count: 32, Full: 0, Almost_Full: 1, Overflow: 0

*** Warning: FIFO is ALMOST FULL! ***

Write: 32, Count: 32, Full: 1, Almost_Full: 1, Overflow: 1
*** FIFO is FULL! No more data can be written! ***
Write: 33, Count: 32, Full: 1, Almost_Full: 1, Overflow: 1
*** FIFO is FULL! No more data can be written! ***
Write: 34, Count: 32, Full: 1, Almost_Full: 1, Overflow: 1
*** FIFO is FULL! No more data can be written! ***
Write: 35, Count: 32, Full: 1, Almost_Full: 1, Overflow: 1
*** FIFO is FULL! No more data can be written! ***
Write: 36, Count: 32, Full: 1, Almost_Full: 1, Overflow: 1
*** FIFO is FULL! No more data can be written! ***

================================================================
======
Read from FIFO until empty to test empty and underflow
================================================================
======

Read: 0, Data_out: 0, Valid: 1, Count: 31, Empty: 0, Almost_Empty: 0,
Underflow: 0
Read: 1, Data_out: 1, Valid: 1, Count: 30, Empty: 0, Almost_Empty: 0,
Underflow: 0
Read: 2, Data_out: 2, Valid: 1, Count: 29, Empty: 0, Almost_Empty: 0,
Underflow: 0
Read: 3, Data_out: 3, Valid: 1, Count: 28, Empty: 0, Almost_Empty: 0,
Underflow: 0
Read: 4, Data_out: 4, Valid: 1, Count: 27, Empty: 0, Almost_Empty: 0,
Underflow: 0
Read: 5, Data_out: 5, Valid: 1, Count: 26, Empty: 0, Almost_Empty: 0,
Underflow: 0
Read: 6, Data_out: 6, Valid: 1, Count: 25, Empty: 0, Almost_Empty: 0,
Underflow: 0
Read: 7, Data_out: 7, Valid: 1, Count: 24, Empty: 0, Almost_Empty: 0,
Underflow: 0
Read: 8, Data_out: 8, Valid: 1, Count: 23, Empty: 0, Almost_Empty: 0,
Underflow: 0
```

```
Read: 9, Data_out: 9, Valid: 1, Count: 22, Empty: 0, Almost_Empty: 0,
Underflow: 0
Read: 10, Data_out: 10, Valid: 1, Count: 21, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 11, Data_out: 11, Valid: 1, Count: 20, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 12, Data_out: 12, Valid: 1, Count: 19, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 13, Data_out: 13, Valid: 1, Count: 18, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 14, Data_out: 14, Valid: 1, Count: 17, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 15, Data_out: 15, Valid: 1, Count: 16, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 16, Data_out: 16, Valid: 1, Count: 15, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 17, Data_out: 17, Valid: 1, Count: 14, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 18, Data_out: 18, Valid: 1, Count: 13, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 19, Data_out: 19, Valid: 1, Count: 12, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 20, Data_out: 20, Valid: 1, Count: 11, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 21, Data_out: 21, Valid: 1, Count: 10, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 22, Data_out: 22, Valid: 1, Count: 9, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 23, Data_out: 23, Valid: 1, Count: 8, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 24, Data_out: 24, Valid: 1, Count: 7, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 25, Data_out: 25, Valid: 1, Count: 6, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 26, Data_out: 26, Valid: 1, Count: 5, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 27, Data_out: 27, Valid: 1, Count: 4, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 28, Data_out: 28, Valid: 1, Count: 3, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 29, Data_out: 29, Valid: 1, Count: 2, Empty: 0, Almost_Empty:
0, Underflow: 0
Read: 30, Data_out: 30, Valid: 1, Count: 1, Empty: 0, Almost_Empty:
1, Underflow: 0
```

```
*** Warning: FIFO is ALMOST EMPTY! ***

Read: 31, Data_out: 31, Valid: 1, Count: 0, Empty: 0, Almost_Empty:
1, Underflow: 0

*** Warning: FIFO is ALMOST EMPTY! ***

Read: 32, Data_out: 31, Valid: 0, Count: 0, Empty: 1, Almost_Empty:
1, Underflow: 1
*** ERROR: FIFO UNDERFLOW occurred! Tried to read when empty! ***
Read: 33, Data_out: 31, Valid: 0, Count: 0, Empty: 1, Almost_Empty:
1, Underflow: 1
*** ERROR: FIFO UNDERFLOW occurred! Tried to read when empty! ***
Read: 34, Data_out: 31, Valid: 0, Count: 0, Empty: 1, Almost_Empty:
1, Underflow: 1
*** ERROR: FIFO UNDERFLOW occurred! Tried to read when empty! ***
Read: 35, Data_out: 31, Valid: 0, Count: 0, Empty: 1, Almost_Empty:
1, Underflow: 1
*** ERROR: FIFO UNDERFLOW occurred! Tried to read when empty! ***
Read: 36, Data_out: 31, Valid: 0, Count: 0, Empty: 1, Almost_Empty:
1, Underflow: 1
*** ERROR: FIFO UNDERFLOW occurred! Tried to read when empty! ***

================================================================
======
Simultaneous write and read
================================================================
======

Simultaneous Write and Read - Data_in: 42, Data_out: 31

Final Flags Empty: 0, Almost_Empty: 1, Full: 0, Almost_Full: 0,
Count: 1

================================================================
======
Partial Write (few locations) and Readback
================================================================
======

Partial Write: 0, Data_in: 100, Count: 2, Full: 0
Partial Write: 1, Data_in: 101, Count: 3, Full: 0
Partial Write: 2, Data_in: 102, Count: 4, Full: 0
```

```
Partial Read: 0, Data_out: 42, Valid: 1, Count: 3, Empty: 0
Partial Read: 1, Data_out: 100, Valid: 1, Count: 2, Empty: 0
Partial Read: 2, Data_out: 101, Valid: 1, Count: 1, Empty: 0
Partial Read: 3, Data_out: 102, Valid: 1, Count: 0, Empty: 0

================================================================
======
FWFT Behavior Check: Write without Read
================================================================
======

FWFT Preload Test | Data Written: 55, Data_out (before rd_en): 55
FWFT Preload Test | After Read, Data_out: 55, Valid: 1

================================================================
======
Test Finished, All Cases for FIFO are Covered
================================================================
======

$finish called from file "FIFO_tb.sv", line 188.
$finish at simulation time              1745000
           V C S   S i m u l a t i o n   R e p o r t
Time: 1745000 ps
CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
Fri May  2 22:42:16 2025
```